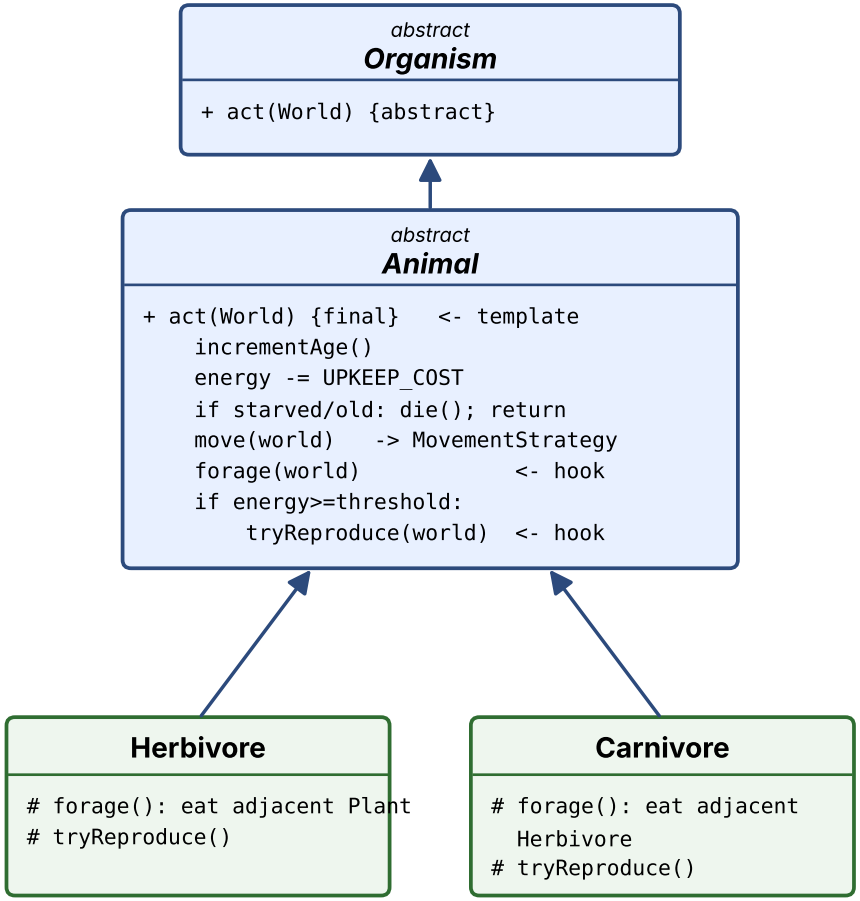
1. Template Method — Animal.act()

src/ecosim/Animal.java (template) · Herbivore.java, Carnivore.java (hooks)



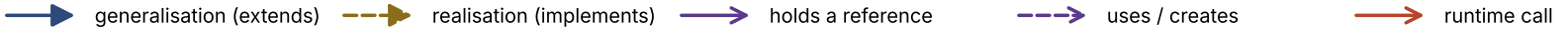
Roles

- AbstractClass = Animal: act() is final and fixes the order of the six steps for every species.
- Primitive operations (hooks) = forage(), tryReproduce(): abstract, filled in by each species.
- ConcreteClasses = Herbivore, Carnivore.

Key interaction

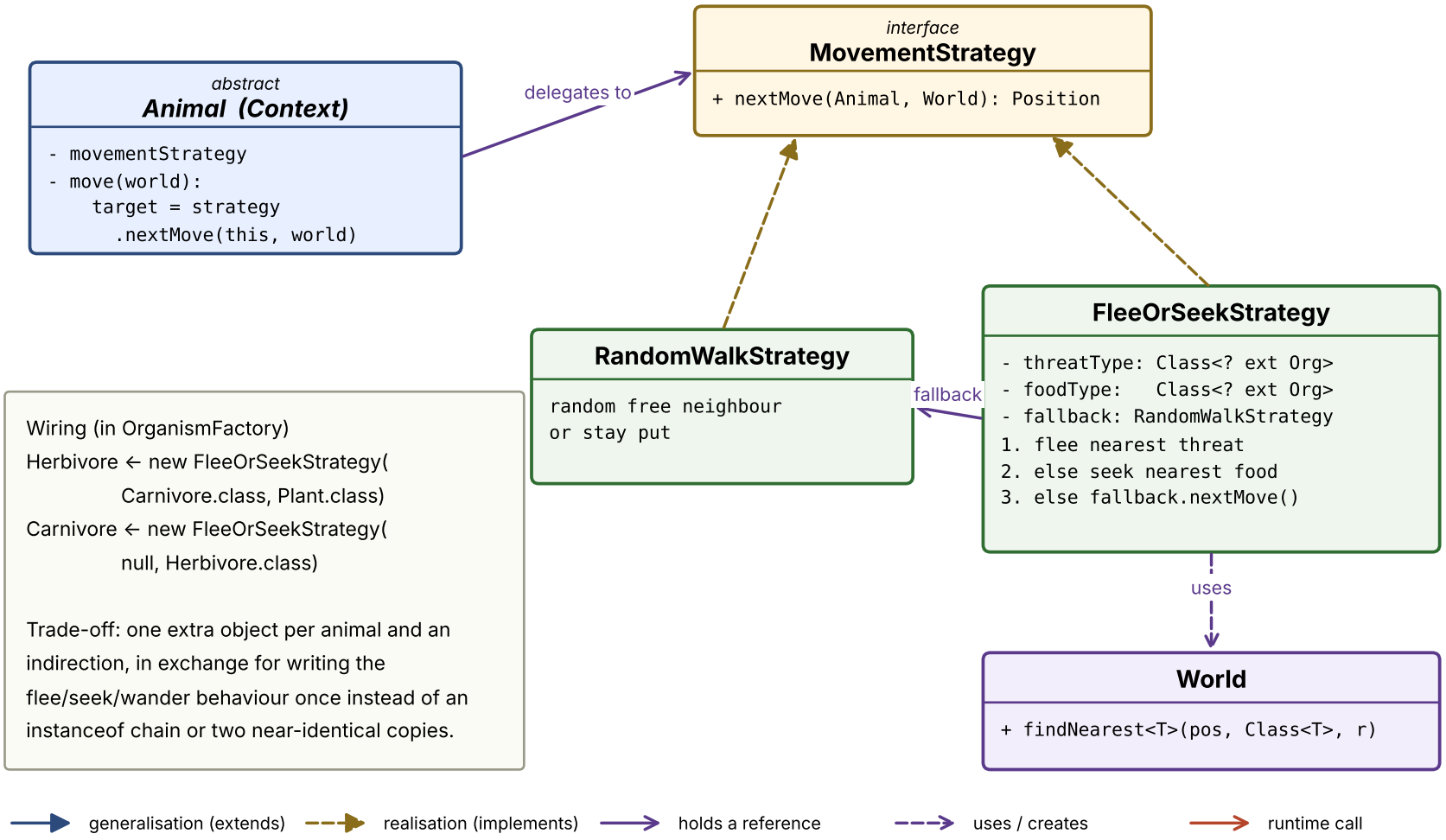
World.tick() calls organism.act(world). The fixed sequence runs in Animal; it calls *down* into the subclass only for the two steps that genuinely differ ("Hollywood principle": don't call us, we'll call you).

Trade-off: subclasses lose the freedom to reorder steps (e.g. eat before moving). Accepted, because ageing/starvation logic written twice would drift out of sync — final makes that impossible.



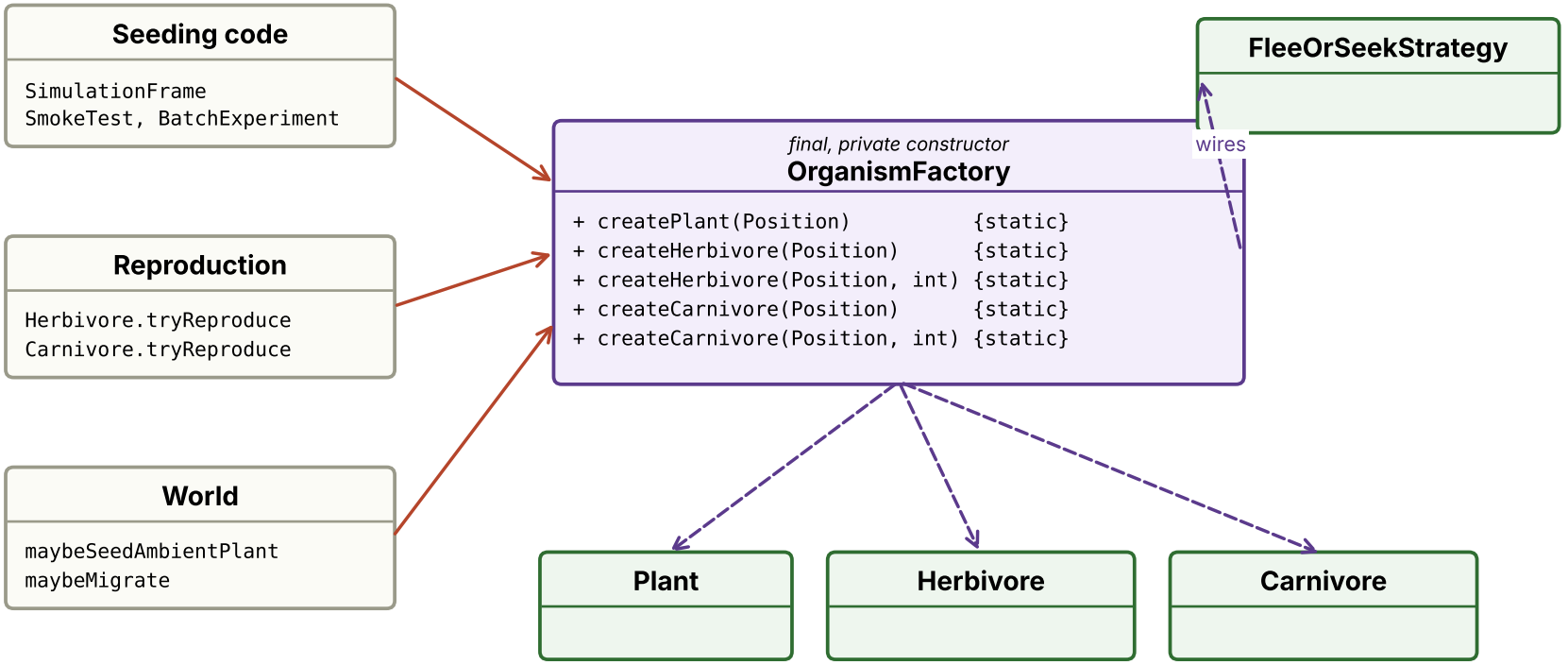
2. Strategy — MovementStrategy

src/ecosim/strategy/ · context: Animal.move()



3. Simple Factory — OrganismFactory

src/ecosim/OrganismFactory.java · clients: SimulationFrame, SmokeTest, BatchExperiment, World, Herbivore, Carnivore



Week-8 change: the (Position, int) overloads let a newborn carry exactly the energy its parent paid, fixing a bug where every birth created energy from nothing. Because construction lived in one place, the fix touched one class plus two call sites.

generalisation (extends) · realisation (implements) · holds a reference · uses / creates · runtime call

4. Observer — SimulationListener (added Weeks 8–10)

src/ecosim/SimulationListener.java · subject: World · observers: SimulationFrame, SmokeTest, BatchExperiment

